```
#--------------------
a=5
b=0

try:
        print("resource open")
        print(a/b)
        k=int(input("enter a number"))
        print(k)
except ZeroDivisionError as e:
        print("the value can not be divided by zero",e)
except ValueError as e:
        print("invalid input")
except Exception as e:
        print("something went wrong...",e)

finally:
        print("resource closed")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex11.py
resource open
the value can not be divided by zero division by zero
resource closed
```

- **Change the value of b to 2 and give the input as some character or string (other than int)**

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/ex12.py
resource open
2.5
enter a number p
invalid input
resource closed
```

Modules (Date, Time, os, calendar, math):

- Modules refer to a file containing Python statements and definitions.

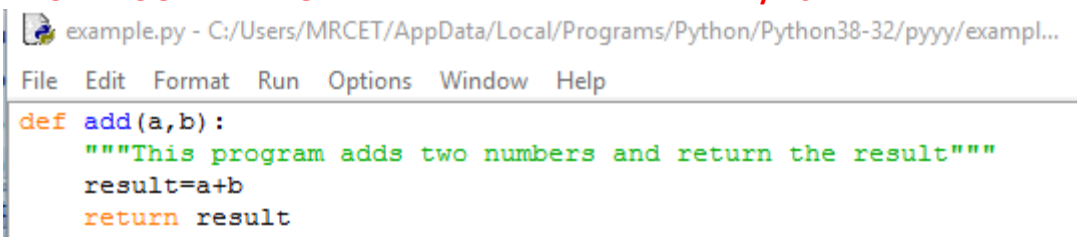
- We use modules to break down large programs into small manageable and organized files. Furthermore, modules provide reusability of code.
- We can define our most used functions in a module and import it, instead of copying their definitions into different programs.
- **Modular programming** refers to the process of breaking a large, unwieldy programming task into separate, smaller, more manageable subtasks or **modules**.

Advantages :

- **Simplicity:** Rather than focusing on the entire problem at hand, a module typically focuses on one relatively small portion of the problem. If you're working on a single module, you'll have a smaller problem domain to wrap your head around. This makes development easier and less error-prone.
- **Maintainability:** Modules are typically designed so that they enforce logical boundaries between different problem domains. If modules are written in a way that minimizes interdependency, there is decreased likelihood that modifications to a single module will have an impact on other parts of the program. This makes it more viable for a team of many programmers to work collaboratively on a large application.
- **Reusability:** Functionality defined in a single module can be easily reused (through an appropriately defined interface) by other parts of the application. This eliminates the need to recreate duplicate code.
- **Scoping:** Modules typically define a separate **namespace**, which helps avoid collisions between identifiers in different areas of a program.
- **Functions**, **modules** and **packages** are all constructs in Python that promote code modularization.

A file containing Python code, for e.g.: example.py, is called a module and its module name would be example.

```
>>> def add(a,b):
        result=a+b
        return result
        """This program adds two numbers and return the result"""“
```



Here, we have defined a function add() inside a module named example. The function takes in two numbers and returns their sum.

How to import the module is:

- We can import the definitions inside a module to another module or the Interactive interpreter in Python.
- We use the import keyword to do this. To import our previously defined module example we type the following in the Python prompt.
- Using the module name we can access the function using dot (.) operation. For Eg:

>>> import example

>>> example.add(5,5)

10

- Python has a ton of standard modules available. Standard modules can be imported the same way as we import our user-defined modules.

Reloading a module:

```
def hi(a,b):
    print(a+b)
hi(4,4)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/add.py

8

>>> import add

8

>>> import add

>>> import add

>>>

Python provides a neat way of doing this. We can use the reload() function inside the imp module to reload a module. This is how its done.

- >>> import imp
- >>> import my_module
- This code got executed >>> import my_module >>> imp.reload(my_module) This code got executed <module 'my_module' from '.\\my_module.py'>how its done.

>>> import imp

>>> import add

>>> imp.reload(add)

8

<module 'add' from 'C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy\\add.py'>

The dir() built-in function

>>> import example

>>> dir(example)

['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'add']

>>> dir()

['__annotations__', '__builtins__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', '__warningregistry__', 'add', 'example', 'hi', 'imp']

It shows all built-in and user-defined modules.

For ex:

```
>>> example.__name__
'example'
```

Datetime module:

 # Write a python program to display date, time

```
>>> import datetime
>>> a=datetime.datetime(2019,5,27,6,35,40)
>>> a
datetime.datetime(2019, 5, 27, 6, 35, 40)
```

 # write a python program to display date

```
import datetime
a=datetime.date(2000,9,18)
print(a)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/d1.py =
2000-09-18
```

 # write a python program to display time

```
import datetime
a=datetime.time(5,3)
print(a)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/d1.py =
05:03:00
```

#write a python program to print date, time for today and now.

```
import datetime
```

```
a=datetime.datetime.today()

b=datetime.datetime.now()

print(a)

print(b)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/d1.py =

2019-11-29 12:49:52.235581

2019-11-29 12:49:52.235581
```

#write a python program to add some days to your present date and print the date added.

```
import datetime

a=datetime.date.today()

b=datetime.timedelta(days=7)

print(a+b)
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/d1.py =

2019-12-06
```

#write a python program to print the no. of days to write to reach your birthday

```
import datetime

a=datetime.date.today()

b=datetime.date(2020,5,27)

c=b-a

print(c)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/d1.py =

180 days, 0:00:00

#write an python program to print date, time using date and time functions

```
import datetime
t=datetime.datetime.today()
print(t.date())
print(t.time())
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/d1.py =

2019-11-29

12:53:39.226763

Time module:

#write a python program to display time.

```
import time
print(time.time())
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/t1.py =

1575012547.1584706

#write a python program to get structure of time stamp.

```
import time
print(time.localtime(time.time()))
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/t1.py =

time.struct_time(tm_year=2019, tm_mon=11, tm_mday=29, tm_hour=13, tm_min=1, tm_sec=15, tm_wday=4, tm_yday=333, tm_isdst=0)

#write a python program to make a time stamp.

```
import time
a=(1999,5,27,7,20,15,1,27,0)
print(time.mktime(a))
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/t1.py =
927769815.0
```

#write a python program using sleep().

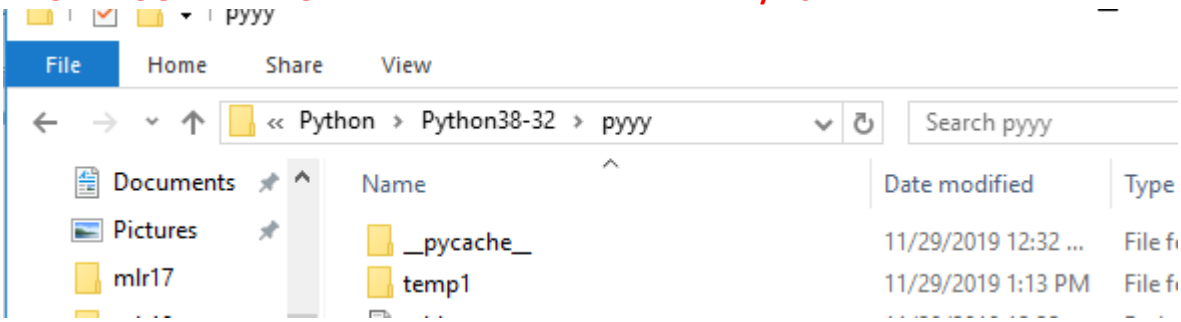
```
import time
time.sleep(6)     #prints after 6 seconds
print("Python  Lab")
```

Output:

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/t1.py =
Python  Lab  (#prints after 6 seconds)
```

os module:

```
>>> import os
>>> os.name
'nt'
>>> os.getcwd()
'C:\\Users\\MRCET\\AppData\\Local\\Programs\\Python\\Python38-32\\pyyy'
>>> os.mkdir("temp1")
```

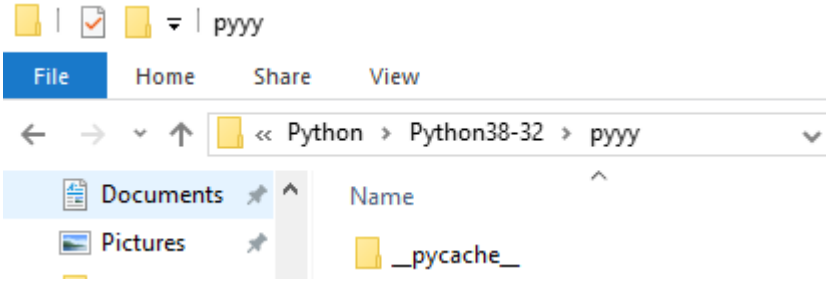


Note: temp1 dir is created

```
>>> os.getcwd()
'C:\\Users\\MRCET\\AppData\\Local\\Programs\\Python\\Python38-32\\pyyy'
>>> open("t1.py","a")
<_io.TextIOWrapper name='t1.py' mode='a' encoding='cp1252'>
>>> os.access("t1.py",os.F_OK)
True
>>> os.access("t1.py",os.W_OK)
True
>>> os.rename("t1.py","t3.py")
>>> os.access("t1.py",os.F_OK)
False
>>> os.access("t3.py",os.F_OK)
True
>>> os.rmdir('temp1')
```

(or)

```
os.rmdir('C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/temp1')
```



Note: Temp1dir is removed

>>> os.remove("t3.py")

Note: We can check with the following cmd whether removed or not

>>> os.access("t3.py",os.F_OK)

False

>>> os.listdir()

['add.py', 'ali.py', 'alia.py', 'arr.py', 'arr2.py', 'arr3.py', 'arr4.py', 'arr5.py', 'arr6.py', 'br.py', 'br2.py', 'bubb.py', 'bubb2.py', 'bubb3.py', 'bubb4.py', 'bubbdesc.py', 'clo.py', 'cmndlinarg.py', 'comm.py', 'con1.py', 'cont.py', 'cont2.py', 'd1.py', 'dic.py', 'e1.py', 'example.py', 'f1.y.py', 'flowof.py', 'fr.py', 'fr2.py', 'fr3.py', 'fu.py', 'fu1.py', 'if1.py', 'if2.py', 'ifelif.py', 'ifelse.py', 'iff.py', 'insertdesc.py', 'inserti.py', 'k1.py', 'l1.py', 'l2.py', 'link1.py', 'linklisttt.py', 'lis.py', 'listlooop.py', 'm1.py', 'merg.py', 'nesforr.py', 'nestedif.py', 'opprec.py', 'paraarg.py', 'qucksort.py', 'qukdesc.py', 'quu.py', 'r.py', 'rec.py', 'ret.py', 'rn.py', 's1.py', 'scoglo.py', 'selecasce.py', 'selectdecs.py', 'stk.py', 'strmodl.py', 'strr.py', 'strr1.py', 'strr2.py', 'strr3.py', 'strr4.py', 'strrmodl.py', 'wh.py', 'wh1.py', 'wh2.py', 'wh3.py', 'wh4.py', 'wh5.py', '__pycache__']

>>> os.listdir('C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32')

['argpar.py', 'br.py', 'bu.py', 'cmndlinarg.py', 'DLLs', 'Doc', 'f1.py', 'f1.txt', 'filess', 'functupretval.py', 'funturet.py', 'gtopt.py', 'include', 'Lib', 'libs', 'LICENSE.txt', 'lisparam.py', 'mysite', 'NEWS.txt', 'niru', 'python.exe', 'python3.dll', 'python38.dll', 'pythonw.exe', 'pyyy', 'Scripts', 'srp.py', 'sy.py', 'symod.py', 'tcl', 'the_weather', 'Tools', 'tupretval.py', 'vcruntime140.dll']

Calendar module:

#write a python program to display a particular month of a year using calendar module.

```
import calendar

print(calendar.month(2020,1))
```

Output:

```
>>>
= RESTART: C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/cl1.py
    January 2020
Mo Tu We Th Fr Sa Su
        1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30 31
```


 # write a python program to check whether the given year is leap or not.

```
import calendar

print(calendar.isleap(2021))
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/cl1.py

False

#write a python program to print all the months of given year.

```
import calendar

print(calendar.calendar(2020,1,1,1))
```

Output:

```
>>>
= RESTART: C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32.
                                2020

      January                   February                  March
Mo Tu We Th Fr Sa Su  Mo Tu We Th Fr Sa Su  Mo Tu We Th Fr Sa Su
       1  2  3  4  5                  1  2                     1
 6  7  8  9 10 11 12   3  4  5  6  7  8  9   2  3  4  5  6  7  8
13 14 15 16 17 18 19  10 11 12 13 14 15 16   9 10 11 12 13 14 15
20 21 22 23 24 25 26  17 18 19 20 21 22 23  16 17 18 19 20 21 22
27 28 29 30 31        24 25 26 27 28 29     23 24 25 26 27 28 29
                                            30 31

      April                     May                       June
Mo Tu We Th Fr Sa Su  Mo Tu We Th Fr Sa Su  Mo Tu We Th Fr Sa Su
       1  2  3  4  5                  1  2  3   1  2  3  4  5  6  7
 6  7  8  9 10 11 12   4  5  6  7  8  9 10   8  9 10 11 12 13 14
13 14 15 16 17 18 19  11 12 13 14 15 16 17  15 16 17 18 19 20 21
20 21 22 23 24 25 26  18 19 20 21 22 23 24  22 23 24 25 26 27 28
27 28 29 30           25 26 27 28 29 30 31  29 30

       July                    August                  September
Mo Tu We Th Fr Sa Su  Mo Tu We Th Fr Sa Su  Mo Tu We Th Fr Sa Su
       1  2  3  4  5                  1  2      1  2  3  4  5  6
 6  7  8  9 10 11 12   3  4  5  6  7  8  9   7  8  9 10 11 12 13
13 14 15 16 17 18 19  10 11 12 13 14 15 16  14 15 16 17 18 19 20
20 21 22 23 24 25 26  17 18 19 20 21 22 23  21 22 23 24 25 26 27
27 28 29 30 31        24 25 26 27 28 29 30  28 29 30
                      31

      October                   November                  December
Mo Tu We Th Fr Sa Su  Mo Tu We Th Fr Sa Su  Mo Tu We Th Fr Sa Su
          1  2  3  4                     1      1  2  3  4  5  6
 5  6  7  8  9 10 11   2  3  4  5  6  7  8   7  8  9 10 11 12 13
12 13 14 15 16 17 18   9 10 11 12 13 14 15  14 15 16 17 18 19 20
19 20 21 22 23 24 25  16 17 18 19 20 21 22  21 22 23 24 25 26 27
26 27 28 29 30 31     23 24 25 26 27 28 29  28 29 30 31
                      30
```


math module:

 # write a python program which accepts the radius of a circle from user and computes the area.

```
import math

r=int(input("Enter radius:"))

area=math.pi*r*r

print("Area of circle is:",area)
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/m.py =

Enter radius:4

Area of circle is: 50.26548245743669

>>> **import math**

>>> **print("The value of pi is", math.pi)**

O/P: The value of pi is 3.141592653589793

Import with renaming:

- We can import a module by renaming it as follows.
- **For Eg:**

  >>> **import math as m**

  >>> **print("The value of pi is", m.pi)**

O/P: The value of pi is 3.141592653589793

- We have renamed the math module as m. This can save us typing time in some cases.
- Note that the name math is not recognized in our scope. Hence, math.pi is invalid, m.pi is the correct implementation.

Python from...import statement:

- We can import specific names from a module without importing the module as a whole. Here is an example.

  >>> from math import pi

  >>> print("The value of pi is", pi)

O/P: The value of pi is 3.141592653589793

- We imported only the attribute pi from the module.
- In such case we don't use the dot operator. We could have imported multiple attributes as follows.

```
>>> from math import pi, e

>>> pi

3.141592653589793

>>> e

2.718281828459045
```

Import all names:

- We can import all names(definitions) from a module using the following construct.

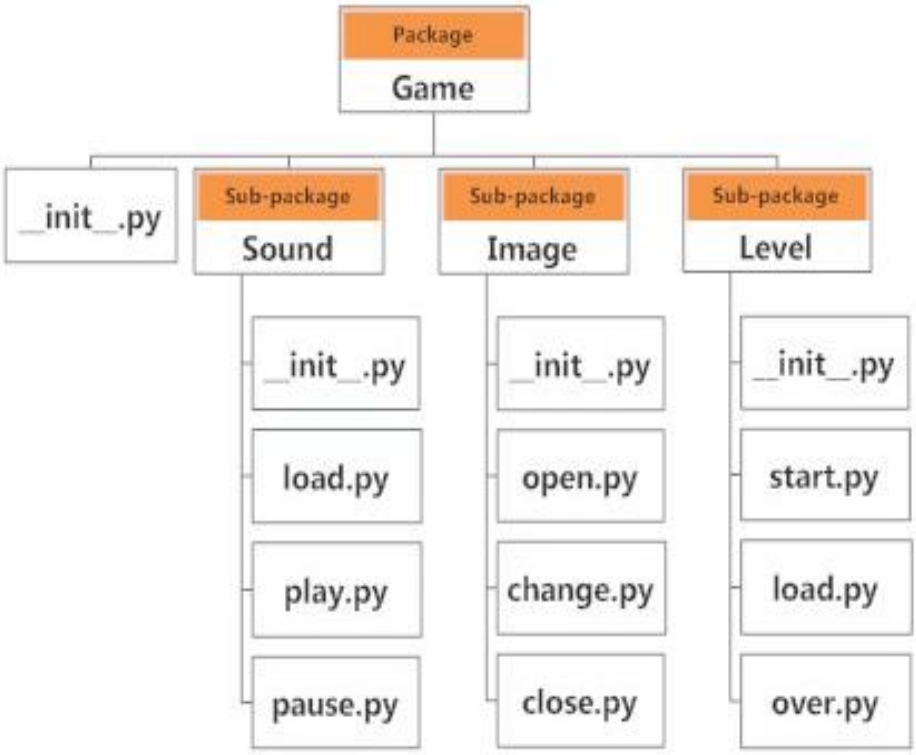
```
>>>from math import *

>>>print("The value of pi is", pi)
```

- We imported all the definitions from the math module. This makes all names except those beginnig with an underscore, visible in our scope.

Explore packages:

- We don't usually store all of our files in our computer in the same location. We use a well-organized hierarchy of directories for easier access.
- Similar files are kept in the same directory, for example, we may keep all the songs in the "music" directory. Analogous to this, Python has packages for directories and modules for files.
- As our application program grows larger in size with a lot of modules, we place similar modules in one package and different modules in different packages. This makes a project (program) easy to manage and conceptually clear.
- Similar, as a directory can contain sub-directories and files, a Python package can have sub-packages and modules.
- A directory must contain a file named __init__.py in order for Python to consider it as a package. This file can be left empty but we generally place the initialization code for that package in this file.
- Here is an example. Suppose we are developing a game, one possible organization of packages and modules could be as shown in the figure below.



- If a file named __init__.py is present in a package directory, it is invoked when the package or a module in the package is imported. This can be used for execution of package initialization code, such as initialization of package-level data.
- For example ***__init__.py***
- A **module** in the package can access the global by importing it in turn
- We can import modules from packages using the dot (.) operator.
- For example, if want to import the start module in the above example, it is done as follows.
- import Game.Level.start
- Now if this module contains a function named **select_difficulty(),** we must use the full name to reference it.
- Game.Level.start.select_difficulty(2)

- If this construct seems lengthy, we can import the module without the package prefix as follows.
- from Game.Level import start
- We can now call the function simply as follows.
- **start.select_difficulty(2)**
- Yet another way of importing just the required function (or class or variable) form a module within a package would be as follows.
- **from Game.Level.start import select_difficulty**
- Now we can directly call this function.
- **select_difficulty(2)**

Examples:

#Write a python program to create a package (II YEAR),sub-package(CSE),modules(student) and create read and write function to module

```
def read():
   print("Department")
def write():
   print("Student")
```

Output:

```
>>> from IIYEAR.CSE import student
>>> student.read()
Department
>>> student.write()
Student
>>> from IIYEAR.CSE.student import read
>>> read
```

<function read at 0x03BD1070>

>>> read()

Department

>>> from IIYEAR.CSE.student import write

>>> write()

Student

 # Write a program to create and import module?

```
def add(a=4,b=6):
    c=a+b
    return c
```

Output:

C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\IIYEAR\modu1.py

>>> from IIYEAR import modu1

>>> modu1.add()

10

 # Write a program to create and rename the existing module.

```
def a():
    print("hello world")
a()
```

Output:

C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/IIYEAR/exam.py

hello world

>>> import exam as ex

hello world